

**Міністерство освіти і науки України**  
**Національний технічний університет України**  
**«Київський політехнічний інститут імені Ігоря Сікорського»**  
**Факультет інформатики та обчислювальної техніки**  
**Кафедра обчислювальної техніки**

**Лабораторна робота №4**  
з дисципліни «Об'єктно-орієнтоване програмування»  
на тему:  
«Вдосконалення структури коду графічного редактора об'єктів»

**Виконав:**

студентка групи ІМ-55

Горбань А.О

Номер в списку: 6

**Перевірив:**

Рекечинський Д.О

### Завдання:

1. Створити у середовищі розробки проект з ім'ям Lab4
2. Написати вихідний текст програми згідно варіанту завдання
3. Скомпілювати вихідний текст і отримати виконуваний файл програми
4. Перевірити роботу програми. Налогодити програму
5. Проаналізувати та прокоментувати результати та вихідний текст програми
6. Оформити звіт

### Постановка завдання (Варіант №6):

1. Динамічний об'єкт **MyEditor**: Створення об'єкта через **new MyEditor()** у повідомленні **WM\_CREATE** та знищення через **delete myEditor** у **WM\_DESTROY**
2. Розмір масиву:  $N = 6 + 100 = 106$  елементів
3. Віртуальне успадкування: Використання **public virtual Shape** для виключення дублювання полів координат **x1, y1, x2, y2** у складних фігурах
4. Складені фігури:
  - **LineOOShape** (Лінія з кружечками): успадковує **LineShape** та **EllipseShape**
  - **CubeShape** (Прозорий каркасний куб): успадковує **RectangleShape** та **LineShape**
5. Інтерфейс: Панель інструментів (Toolbar) на 6 кнопок, режим «гумового джгута» та вікно «Про програму»

### Файл resource.h

```
#ifndef RESOURCE_H
#define RESOURCE_H

#define IDR_MAIN_MENU    101
#define IDB_TOOLBAR      102

#define IDM_POINT        201
#define IDM_LINE         202
#define IDM_RECTANGLE    203
#define IDM_ELLIPSE      204
#define IDM_LINEOO       205
#define IDM_CUBE         206
#define IDM_EXIT         207
```

```
#define IDM_ABOUT      208

#endif
```

### Файл resource.rc

```
#include <windows.h>
```

```
#include "resource.h"
```

```
#pragma code_page(65001)
```

```
IDR_MAIN_MENU MENU
```

```
BEGIN
```

```
POPUP "Файл"
```

```
BEGIN
```

```
    MENUITEM "Вихід", IDM_EXIT
```

```
END
```

```
POPUP "Об'єкти"
```

```
BEGIN
```

```
    MENUITEM "Крапка", IDM_POINT
```

```
    MENUITEM "Лінія", IDM_LINE
```

```
    MENUITEM "Прямокутник", IDM_RECTANGLE
```

```
    MENUITEM "Еліпс", IDM_ELLIPSE
```

```
    MENUITEM "Лінія з кружечками", IDM_LINEOO
```

```
    MENUITEM "Куб", IDM_CUBE
```

```
END
```

POPUP "Довідка"

BEGIN

MENUITEM "Про програму", IDM\_ABOUT

END

END

IDB\_TOOLBAR BITMAP "toolbar.bmp"

### Файл shape.h

```
#ifndef SHAPE_H
#define SHAPE_H

#include <windows.h>
#include <cmath>

class Shape {
protected:
    long x1, y1, x2, y2;
public:
    virtual ~Shape() {}

    virtual void SetPoints(long startX, long startY, long endX, long endY) {
        x1 = startX; y1 = startY;
        x2 = endX;    y2 = endY;
    }

    virtual void Draw(HDC hdc) = 0;
};
```

```
};
```

```
class PointShape : public virtual Shape {
```

```
public:
```

```
    void Draw(HDC hdc) override {
```

```
        SetPixel(hdc, x1, y1, RGB(0, 0, 0));
```

```
    }
```

```
};
```

```
class LineShape : public virtual Shape {
```

```
public:
```

```
    void Draw(HDC hdc) override {
```

```
        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
```

```
        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
```

```
        MoveToEx(hdc, x1, y1, NULL);
```

```
        LineTo(hdc, x2, y2);
```

```
        SelectObject(hdc, hOldPen);
```

```
        DeleteObject(hPen);
```

```
    }
```

```
};
```

```
class RectangleShape : public virtual Shape {
```

```
public:

    void Draw(HDC hdc) override {

        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HBRUSH hBrush = CreateSolidBrush(RGB(144, 238, 144)); // Світло-зелене
заповнення

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        Rectangle(hdc, x1, y1, x2, y2);

        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

        DeleteObject(hBrush);

    }

};
```

```
class EllipseShape : public virtual Shape {

public:

    void Draw(HDC hdc) override {

        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
```

```
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

        Ellipse(hdc, x1, y1, x2, y2);

        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

    }

};
```

```
class LineOOShape : public LineShape, public EllipseShape {
public:

    void Draw(HDC hdc) override {

        LineShape::Draw(hdc);

        long ox1 = x1, oy1 = y1, ox2 = x2, oy2 = y2;

        int r = 5;

        SetPoints(ox1 - r, oy1 - r, ox1 + r, oy1 + r);

        EllipseShape::Draw(hdc);

        SetPoints(ox2 - r, oy2 - r, ox2 + r, oy2 + r);

        EllipseShape::Draw(hdc);
    }
};
```

```

        SetPoints(ox1, oy1, ox2, oy2);

    }

};

class CubeShape : public RectangleShape, public LineShape {
public:

    void Draw(HDC hdc) override {

        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH); // Прозора кисть

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

        int dx = (x2 - x1) / 3;

        int dy = (y2 - y1) / 3;

        // Задній прямокутник

        Rectangle(hdc, x1 + dx, y1 - dy, x2 + dx, y2 - dy);

        // Передній прямокутник

        Rectangle(hdc, x1, y1, x2, y2);

        // З'єднувальні лінії кутів

        MoveToEx(hdc, x1, y1, NULL); LineTo(hdc, x1 + dx, y1 - dy);
    }
};

```

```

        MoveToEx(hdc, x2, y1, NULL); LineTo(hdc, x2 + dx, y1 - dy);

        MoveToEx(hdc, x1, y2, NULL); LineTo(hdc, x1 + dx, y2 - dy);

        MoveToEx(hdc, x2, y2, NULL); LineTo(hdc, x2 + dx, y2 - dy);


        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

    }

};

#endif

```

## Файл editor.h

```

#ifndef EDITOR_H

#define EDITOR_H

#ifndef _WIN32_IE

#define _WIN32_IE 0x0500

#endif

#include <windows.h>

#include <commctrl.h>

#include "shape.h"

#include "resource.h"

```

```
const int MAX_SHAPES = 106; // Вариант 6: N = 6 + 100 = 106
```

```
class MyEditor {
```

```
private:
```

```
    Shape* pcshape[MAX_SHAPES];
```

```
    int shapeCount;
```

```
    int currentTool;
```

```
    bool isDrawing;
```

```
    POINT startPoint;
```

```
    POINT currentPoint;
```

```
    HWND hWndParent;
```

```
    HWND hToolbar;
```

```
    HPEN hRubberPen;
```

```
    void UpdateWindowTitle();
```

```
    void DrawRubberBand(HWND hWnd, POINT pt1, POINT pt2);
```

```
public:
```

```
    MyEditor();
```

```
~MyEditor();

void Init(HWND hWnd);

void OnSize();

void OnCommand(WPARAM wParam);

LRESULT OnNotify(LPARAM lParam);

void OnLButtonDown(HWND hWnd, LPARAM lParam);

void OnMouseMove(HWND hWnd, LPARAM lParam);

void OnLButtonUp(HWND hWnd, LPARAM lParam);

void OnPaint(HWND hWnd);

};

#endif
```

## Файл editor.cpp

```
#include "editor.h"

MyEditor::MyEditor() {

    shapeCount = 0;

    currentTool = IDM_POINT;

    isDrawing = false;

    hToolbar = NULL;
```

```
hWndParent = NULL;

hRubberPen = NULL;

for (int i = 0; i < MAX_SHAPES; ++i) {

    pcshape[i] = NULL;

}

}
```

```
MyEditor::~MyEditor() {

    for (int i = 0; i < shapeCount; ++i) {

        if (pcshape[i]) {

            delete pcshape[i];

            pcshape[i] = NULL;

        }

    }

    if (hRubberPen) {

        DeleteObject(hRubberPen);

    }

}
```

```
void MyEditor::Init(HWND hWnd) {

    hWndParent = hWnd;
```

```
INITCOMMONCONTROLSEX icex;

icex.dwSize = sizeof(INITCOMMONCONTROLSEX);

icex.dwICC = ICC_BAR_CLASSES;

InitCommonControlsEx(&icex);


hToolbar = CreateToolbarEx(

    hWnd,

    WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS,

    1, 6, GetModuleHandle(NULL), IDB_TOOLBAR,

    NULL, 0, 16, 16, 16, 16, sizeof(TBBUTTON)

);


if (hToolbar) {

    SendMessage(hToolbar, TB_BUTTONSTRUCTSIZE, (WPARAM) sizeof(TBBUTTON), 0);


    TBBUTTON tbb[6] = {

        { 0, IDM_POINT,      TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },

        { 1, IDM_LINE,       TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },

        { 2, IDM_RECTANGLE,  TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },

        { 3, IDM_ELLIPSE,    TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },

        { 4, IDM_LINEOO,     TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 },
```

```

        { 5, IDM_CUBE,          TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0 }

};

SendMessage(hToolbar, TB_ADDBUTTONS, 6, (LPARAM)&tbb);

SendMessage(hToolbar, TB_AUTOSIZE, 0, 0);

ShowWindow(hToolbar, SW_SHOW);

}

UpdateWindowTitle();

}

void MyEditor::OnSize() {

    if (hToolbar) {

        SendMessage(hToolbar, TB_AUTOSIZE, 0, 0);

    }

}

void MyEditor::UpdateWindowTitle() {

    const wchar_t* toolName = L"Крaпка";

    switch (currentTool) {

        case IDM_POINT:      toolName = L"Крaпка"; break;

        case IDM_LINE:       toolName = L"Лiнiя"; break;

```

```
        case IDM_RECTANGLE: toolName = L"Прямокутник"; break;

        case IDM_ELLIPSE:   toolName = L"Еліпс"; break;

        case IDM_LINEOO:    toolName = L"Лінія з кружечками"; break;

        case IDM_CUBE:      toolName = L"Куб"; break;

    }

    wchar_t title[128];

    wsprintf(title, L"Лабораторна 4 - [Поточний інструмент: %s]", toolName);

    SetWindowText(hWndParent, title);
}

void MyEditor::OnCommand(WPARAM wParam) {

    int id = LOWORD(wParam);

    if (id >= IDM_POINT && id <= IDM_CUBE) {

        currentTool = id;

        UpdateWindowTitle();

    } else if (id == IDM_ABOUT) {

        MessageBox(hWndParent, L"Лабораторна робота №4\nТема: Ієрархія класів та віртуальне успадкування", L"Про програму", MB_OK | MB_ICONINFORMATION);

    } else if (id == IDM_EXIT) {

        PostQuitMessage(0);

    }
}
```

```
}
```

```
LRESULT MyEditor::OnNotify(LPARAM lParam) {
```

```
    LPNMHDR pnmh = (LPNMHDR)lParam;
```

```
    if (pnmh->code == TTN_GETDISPINFO) {
```

```
        NMTTDISPINFOW* pttdi = (NMTTDISPINFOW*)lParam;
```

```
        switch (pttdi->hdr.idFrom) {
```

```
            case IDM_POINT:      pttdi->lpszText = (LPWSTR)L"Малювати крапку";  
break;
```

```
            case IDM_LINE:       pttdi->lpszText = (LPWSTR)L"Малювати лінію";  
break;
```

```
            case IDM_RECTANGLE:  pttdi->lpszText = (LPWSTR)L"Малювати  
прямокутник"; break;
```

```
            case IDM_ELLIPSE:    pttdi->lpszText = (LPWSTR)L"Малювати еліпс";  
break;
```

```
            case IDM_LINEOO:     pttdi->lpszText = (LPWSTR)L"Малювати лінію з  
кружечками"; break;
```

```
            case IDM_CUBE:       pttdi->lpszText = (LPWSTR)L"Малювати каркас  
куба"; break;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
void MyEditor::OnLButtonDown(HWND hWnd, LPARAM lParam) {
```

```

        isDrawing = true;

        startPoint.x = LOWORD(lParam);

        startPoint.y = HIWORD(lParam);

        currentPoint = startPoint;

        hRubberPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));

        SetCapture(hWnd);
    }

void MyEditor::DrawRubberBand(HWND hWnd, POINT pt1, POINT pt2) {

    HDC hdc = GetDC(hWnd);

    int oldROP = SetROP2(hdc, R2_NOTXORPEN);

    HPEN hOldPen = (HPEN)SelectObject(hdc, hRubberPen);

    HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

    if (currentTool == IDM_LINE || currentTool == IDM_LINEOO) {

        MoveToEx(hdc, pt1.x, pt1.y, NULL);

        LineTo(hdc, pt2.x, pt2.y);

    } else if (currentTool == IDM_RECTANGLE || currentTool == IDM_CUBE) {

        Rectangle(hdc, pt1.x, pt1.y, pt2.x, pt2.y);
    }
}

```

```

    } else if (currentTool == IDM_ELLIPSE) {

        Ellipse(hdc, pt1.x, pt1.y, pt2.x, pt2.y);

    }

    SelectObject(hdc, hOldBrush);

    SelectObject(hdc, hOldPen);

    SetROP2(hdc, oldROP);

    ReleaseDC(hWnd, hdc);
}

void MyEditor::OnMouseMove(HWND hWnd, LPARAM lParam) {

    if (!isDrawing || currentTool == IDM_POINT) return;

    DrawRubberBand(hWnd, startPoint, currentPoint);

    currentPoint.x = LOWORD(lParam);

    currentPoint.y = HIWORD(lParam);

    DrawRubberBand(hWnd, startPoint, currentPoint);

}

void MyEditor::OnLButtonUp(HWND hWnd, LPARAM lParam) {

    if (!isDrawing) return;

```

```
if (currentTool != IDM_POINT) {

    DrawRubberBand(hWnd, startPoint, currentPoint);

}

ReleaseCapture();

isDrawing = false;

if (hRubberPen) {

    DeleteObject(hRubberPen);

    hRubberPen = NULL;

}

if (shapeCount >= MAX_SHAPES) return;

POINT endPoint = { LOWORD(lParam), HIWORD(lParam) };

Shape* newShape = NULL;

switch (currentTool) {

    case IDM_POINT:      newShape = new PointShape(); break;

    case IDM_LINE:       newShape = new LineShape(); break;

    case IDM_RECTANGLE:  newShape = new RectangleShape(); break;

    case IDM_ELLIPSE:    newShape = new EllipseShape(); break;
```

```

        case IDM_LINEOO:    newShape = new LineOOShape(); break;

        case IDM_CUBE:      newShape = new CubeShape(); break;

    }

    if (newShape) {

        newShape->SetPoints(startPoint.x, startPoint.y, endPoint.x, endPoint.y);

        pcshape[shapeCount++] = newShape;

        InvalidateRect(hWnd, NULL, TRUE);

    }
}

void MyEditor::OnPaint(HWND hWnd) {

    PAINTSTRUCT ps;

    HDC hdc = BeginPaint(hWnd, &ps);

    for (int i = 0; i < shapeCount; ++i) {

        if (pcshape[i]) {

            pcshape[i]->Draw(hdc);

        }

    }

    EndPaint(hWnd, &ps);
}

```

```
}
```

## Файл main.cpp

```
#include <windows.h>

#include <commctrl.h>

#include "editor.h"

#include "resource.h"


// Динамічний вказівник на об'єкт редактуру за вимогами парного варіанта
(Варіант 6)

MyEditor* myEditor = NULL;


LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{

    switch (message) {

        case WM_CREATE:

            myEditor = new MyEditor(); // Динамічне створення

            if (myEditor) {

                myEditor->Init(hWnd);

            }

            break;


        case WM_SIZE:

            if (myEditor) myEditor->OnSize();
```

```
        break;

case WM_COMMAND:

    if (myEditor) myEditor->OnCommand(wParam);

    break;

case WM_NOTIFY:

    if (myEditor) return myEditor->OnNotify(lParam);

    break;

case WM_LBUTTONDOWN:

    if (myEditor) myEditor->OnLButtonDown(hWnd, lParam);

    break;

case WM_MOUSEMOVE:

    if (myEditor) myEditor->OnMouseMove(hWnd, lParam);

    break;

case WM_LBUTTONUP:

    if (myEditor) myEditor->OnLButtonUp(hWnd, lParam);

    break;
```

```

        case WM_PAINT:

            if (myEditor) myEditor->OnPaint(hWnd);

            break;

        case WM_DESTROY:

            if (myEditor) {

                delete myEditor; // Коректне знищення динамічного об'єкта

                myEditor = NULL;

            }

            PostQuitMessage(0);

            break;

        default:

            return DefWindowProc(hWnd, message, wParam, lParam);

    }

    return 0;
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR
lpCmdLine, int nCmdShow) {

    WNDCLASSEX wc = { sizeof(WNDCLASSEX) };

    wc.style          = CS_HREDRAW | CS_VREDRAW;

```

```
wc.lpfnWndProc    = WndProc;

wc.hInstance      = hInstance;

wc.hCursor        = LoadCursor(NULL, IDC_ARROW);

wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

wc.lpszMenuName   = MAKEINTRESOURCE(IDR_MAIN_MENU);

wc.lpszClassName = L"Lab4WindowClass";


RegisterClassEx(&wc);


HWND hWnd = CreateWindowEx(

    0, L"Lab4WindowClass", L"Лабораторна 4",

    WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,

    800, 600, NULL, NULL, hInstance, NULL

);


ShowWindow(hWnd, nCmdShow);

UpdateWindow(hWnd);


MSG msg;

while (GetMessage(&msg, NULL, 0, 0)) {

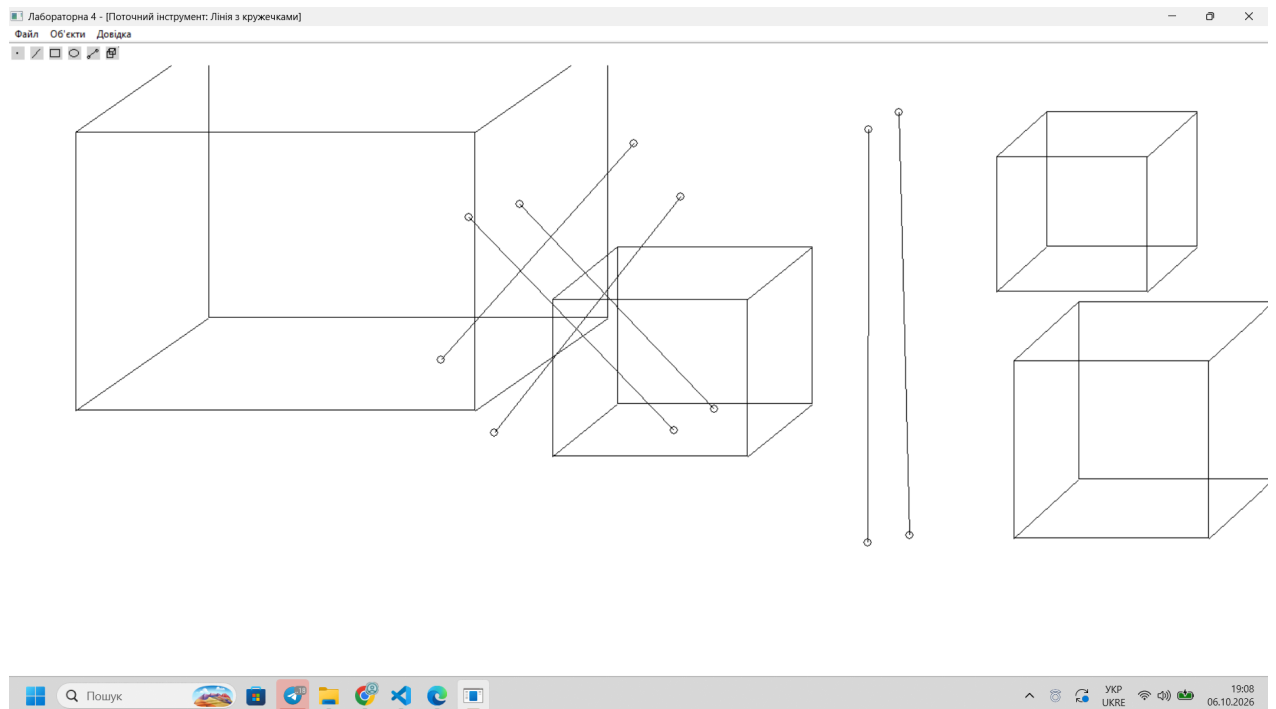
    TranslateMessage(&msg);

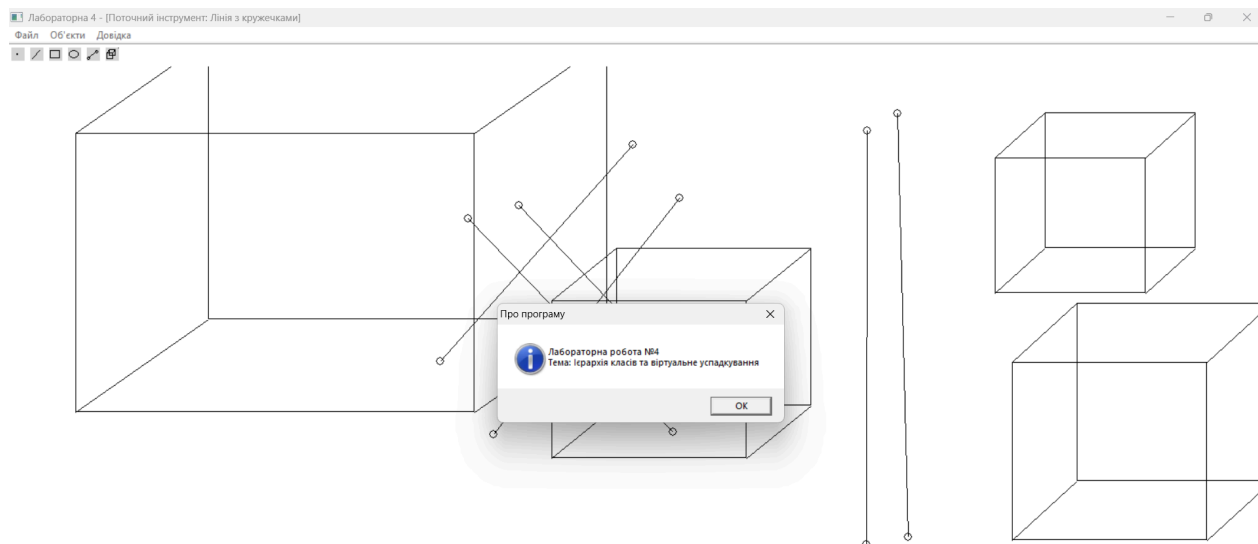
    DispatchMessage(&msg);
```

```
}
```

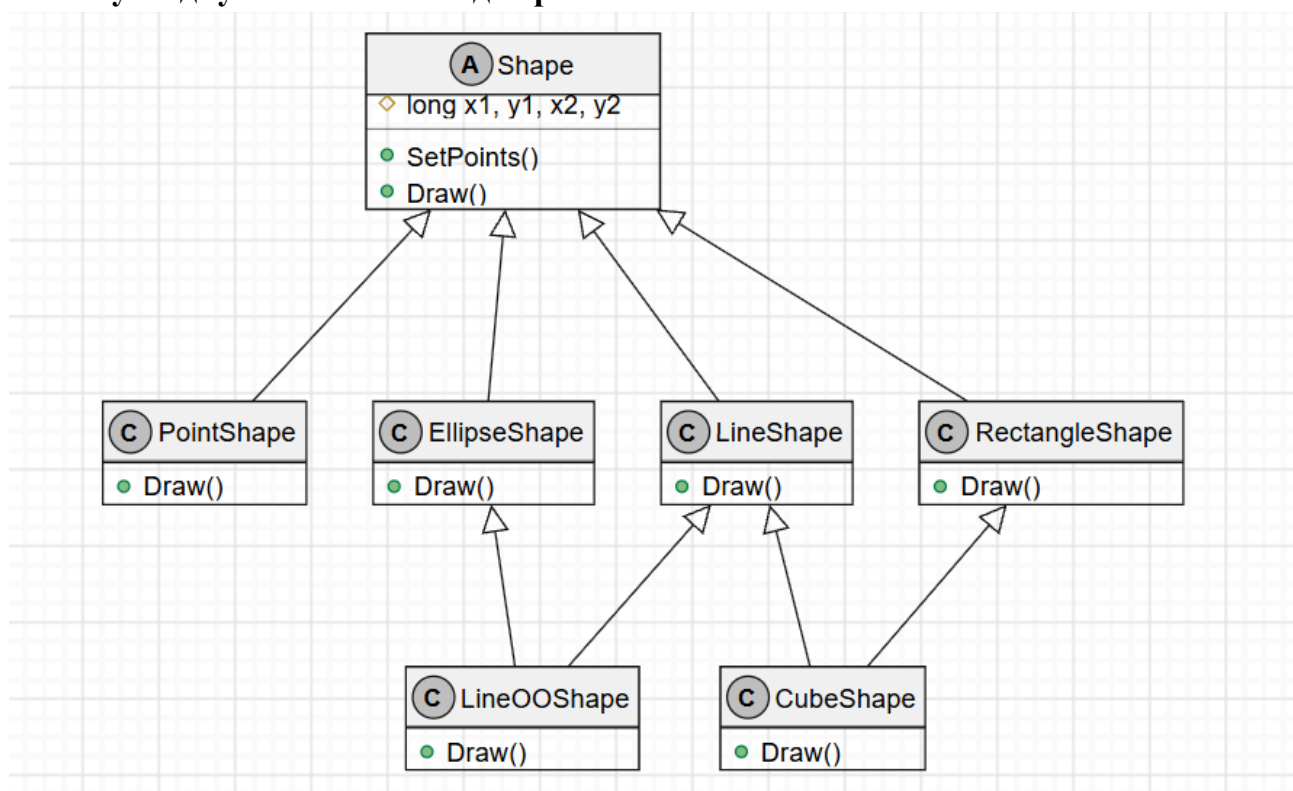
```
return (int)msg.wParam;
```

## Скріншоти:





### Схема успадкування класів – діаграма класів:



### Висновки:

Під час виконання лабораторної роботи №4 було опановано використання віртуального успадкування (**public virtual**) для розв'язання проблеми ромбоподібного успадкування (Diamond Problem) при формуванні складних графічних класів (**CubeShape**, **LineOOShape**). Також було повністю виконано вимогу парного варіанта (Варіант №6) щодо динамічного керування об'єктом головного редактора **MyEditor** через оператори **new** та **delete** в циклі обробки Win32 повідомлень